

DBSCAN

DBSCAN

Density-Based Spatial Clustering
of Applications with Noise

Discover clusters of arbitrary shapes · Detect outliers automatically · No K required

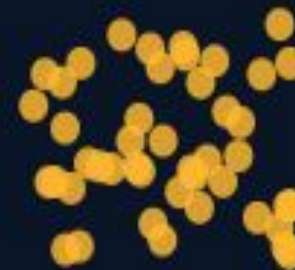
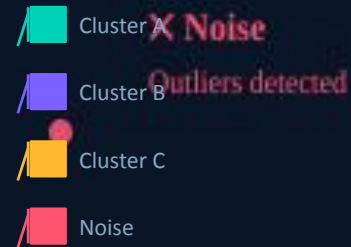
Unsupervised

Clustering

Scikit-learn

Anomaly Detection

Spatial Data



What We'll Cover Today

Eight topics from theory to production-ready code

01

The Clustering Problem

Why shape and noise matter

02

DBSCAN Core Concepts

ϵ , MinPts, core/border/noise points

03

The Algorithm Step-by-Step

How DBSCAN expands clusters

04

Choosing ϵ and MinPts

k-distance graph & rules of thumb

05

DBSCAN vs K-Means vs HDBSCAN

Comparison, strengths, limitations

06

Implementation in Python

Scikit-learn, visualisation, metrics

07

Limitations & When to Use

High dimensions, varying density

08

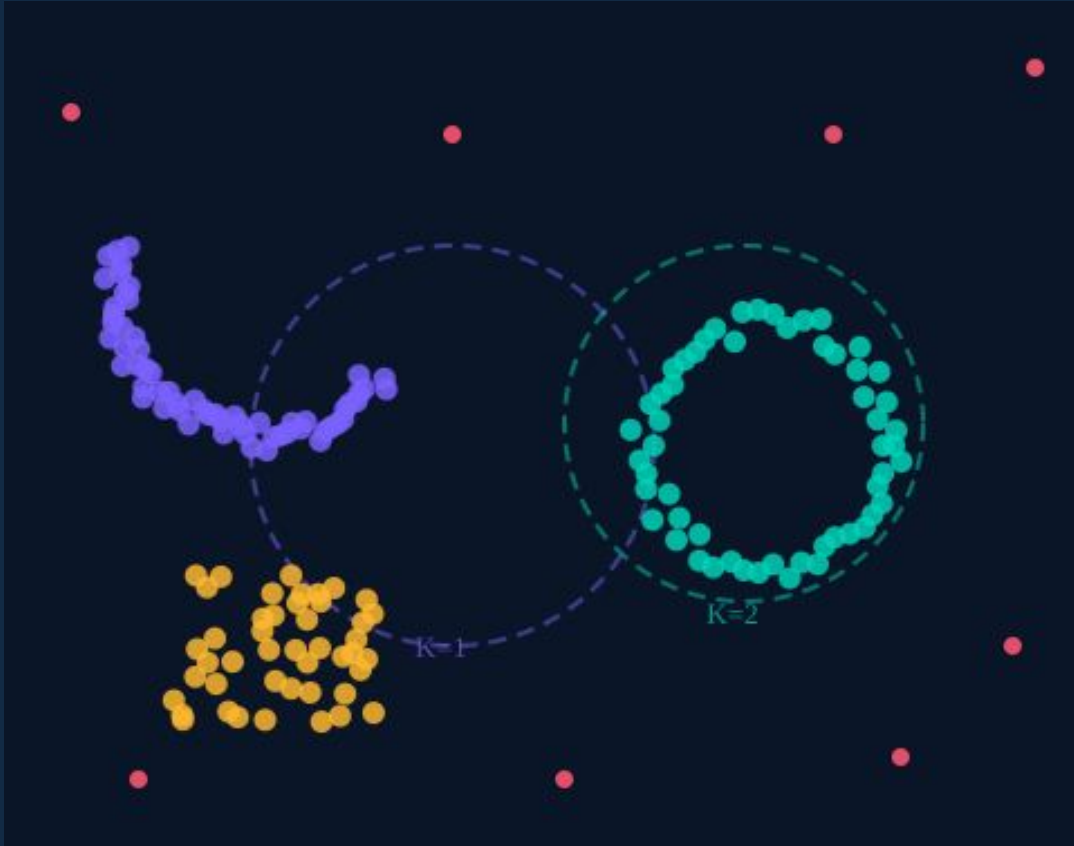
Real-World Applications

Geospatial, fraud, medical, astronomy

The Clustering Problem

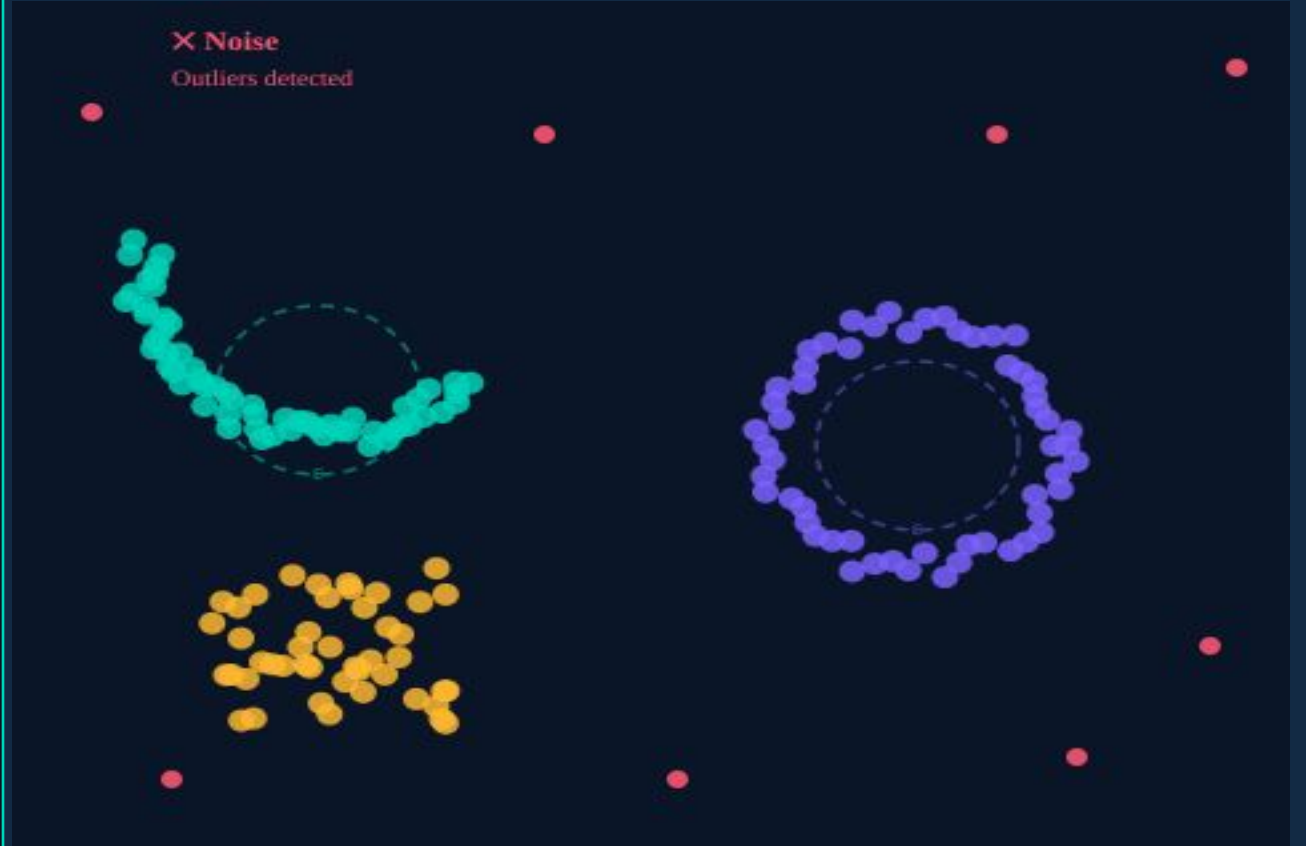
Not all clusters are round — and real data is messy

K-Means Fails on Non-Convex Shapes



K-Means forces spherical clusters — it misclassifies crescent and ring shapes.

DBSCAN Gets It Right



DBSCAN finds arbitrary shapes and marks genuine outliers as noise (red X).

DBSCAN Core Concepts

Two parameters, three point types — that's all you need

ϵ (epsilon)

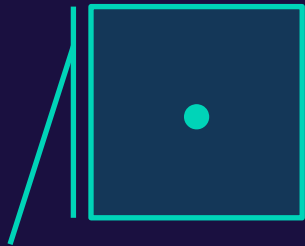
Neighbourhood radius.

Any two points within ϵ of each other are considered neighbours.

MinPts

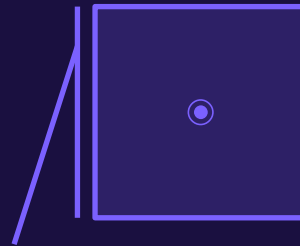
Minimum neighbours required within ϵ to qualify as a core point.

Controls cluster density threshold.



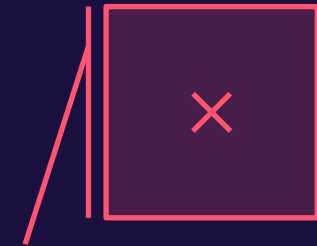
CORE Point

Has $\geq$ MinPts neighbours within ϵ .
Forms the dense interior of a cluster.
All clusters are grown FROM core points.



BORDER Point

Has $<$ MinPts neighbours within ϵ
but falls within ϵ of a core point.
Belongs to a cluster but is not a seed.



NOISE Point

Has $<$ MinPts neighbours within ϵ
and is NOT within ϵ of any core point.
Also called outlier — assigned label -1 .



Key concept: Density-reachability is transitive — if A can reach B and B can reach C via core points, then A and C are in the SAME cluster.

The Algorithm — Step by Step

How DBSCAN expands clusters from dense seeds

```
DBSCAN(D,  $\epsilon$ , MinPts):
    label all points as UNVISITED
    for each unvisited point P:
        mark P as VISITED
        N = neighbourhood(P,  $\epsilon$ )
        if |N| < MinPts:
            label P as NOISE
        else:
            C = new cluster
            expand_cluster(P, N, C,  $\epsilon$ , MinPts)

expand_cluster(P, N, C,  $\epsilon$ , MinPts):
    add P to cluster C
    for each point Q in N:
        if Q is UNVISITED:
            mark Q as VISITED
            N' = neighbourhood(Q,  $\epsilon$ )
            if |N'| >= MinPts:
                N = N  $\cup$  N' # expand!
    if Q not in any cluster:
        add Q to cluster C
```

1

Initialise

All points start as UNVISITED. Pick any unvisited point.

2

Query ϵ -ball

Find all neighbours within radius ϵ of the current point.

3

Core Check

If fewer than MinPts neighbours $\rightarrow$ label as NOISE (may change later).

4

Seed New Cluster

If $\geq$ MinPts neighbours $\rightarrow$ this is a CORE point. Start a new cluster.

5

Expand BFS/DFS

Add all ϵ -neighbours to the cluster. For each one that is also a core point, add ITS neighbours too.

6

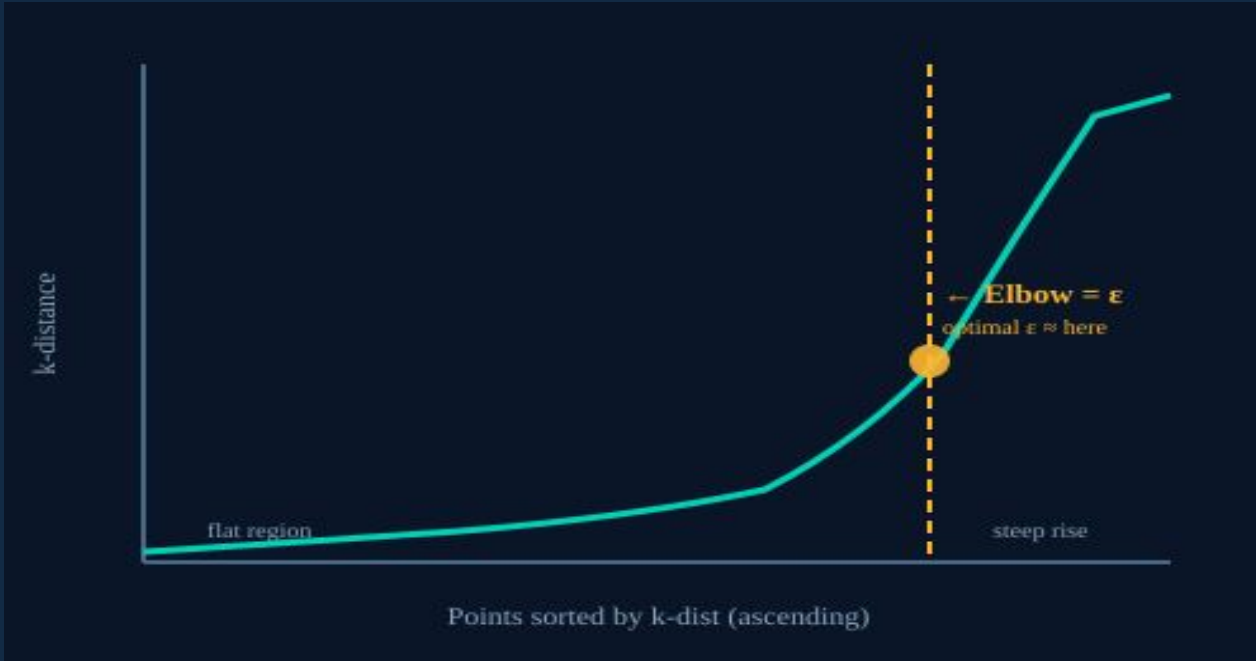
Repeat

Continue until all points visited. Noise points not reachable from any core remain labeled -1.

Choosing ϵ and MinPts

The *k*-distance graph method + practical rules of thumb

The k-Distance Graph Method



1. Choose $k = \text{MinPts} - 1$
2. For each point, compute its k -th nearest-neighbour distance
3. Sort distances ascending and plot
4. Find the 'elbow' — that distance is your ϵ

MinPts Rule

MinPts $\geq$ dimensionality + 1
For 2D data: MinPts = 4 (minimum)
For higher dims: MinPts = $2 \times \text{dims}$
Larger MinPts = more robust to noise

ϵ Rule

Use k -distance graph (left).
Small $\epsilon \rightarrow$ many small clusters + noise
Large $\epsilon \rightarrow$ one giant cluster
Normalise features before picking ϵ !

Effect of Parameters

Too small $\epsilon \rightarrow$ everything is noise
Too large $\epsilon \rightarrow$ one big cluster
Too small MinPts $\rightarrow$ noisy clusters
Too large MinPts $\rightarrow$ core points missed

Practical Starting Point

Start: MinPts = $2 \times n_{\text{features}}$
Run k -distance on scaled data
Pick ϵ at the elbow
Iterate with silhouette score

DBSCAN vs K-Means vs HDBSCAN

Choosing the right clustering algorithm for your task

Feature	K-Means	DBSCAN	HDBSCAN
Need to specify K?	Yes ✕	No ✓	No ✓
Cluster shapes	Convex only	Arbitrary	Arbitrary
Handles outliers?	No ✕	Yes ✓	Yes ✓
Varying density clusters?	No ✕	Struggles ✕	Yes ✓
Speed (large data)	Fast $O(nk)$	$O(n \log n)^*$	$O(n \log n)^*$
Noise label output	None	Label = -1	Label = -1
Reproducible?	Initialisation-sensitive	Yes	Yes
Best for	Spherical blobs	Spatial/geo	Multi-density

* With spatial index (KD-tree / Ball-tree). Without index: $O(n^2)$. Use algorithm='ball_tree' in scikit-learn for large datasets.

Implementation in Python

Scikit-learn DBSCAN — from data to labelled clusters in 10 lines

```
import numpy as np
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import make_moons
import matplotlib.pyplot as plt

# 1. Generate non-convex dataset
X, _ = make_moons(n_samples=300, noise=0.08)

# 2. CRITICAL: Scale features before DBSCAN
X_scaled = StandardScaler().fit_transform(X)

# 3. Fit DBSCAN
db = DBSCAN(eps=0.3, min_samples=5)
labels = db.fit_predict(X_scaled)

# 4. Analyse results
n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
n_noise = list(labels).count(-1)
print(f'Clusters: {n_clusters}, Noise: {n_noise}')
```

```
# 5. Identify point types
core_mask = np.zeros_like(labels, dtype=bool)
core_mask[db.core_sample_indices_] = True

# 6. Visualise
unique_labels = set(labels)
colors = plt.cm.tab10(np.linspace(0, 1, len(unique_labels)))

for k, col in zip(unique_labels, colors):
    if k == -1: col = 'red' # noise
    mask = labels == k
    # Core points – large markers
    plt.scatter(X_scaled[mask & core_mask, 0],
                X_scaled[mask & core_mask, 1],
                c=[col], s=50, edgecolors='k')
    # Border/noise – small markers
    plt.scatter(X_scaled[mask & ~core_mask, 0],
                X_scaled[mask & ~core_mask, 1],
                c=[col], s=12, alpha=0.6)
plt.title('DBSCAN Clusters'); plt.show()
```



Scale First!

ALWAYS StandardScaler before DBSCAN.
It's distance-based — unscaled features ruin it.

core_sample_indices_

Access via `db.core_sample_indices_`.
Core points are the cluster seeds.

Metric param

Default is 'euclidean'.
Try 'cosine' for text, 'haversine' for GPS coords.

Evaluating DBSCAN Results

Internal metrics when ground truth is unavailable

```
from sklearn.metrics import (
    silhouette_score,
    davies_bouldin_score,
    calinski_harabasz_score
)

# Exclude noise points for metrics
mask = labels != -1
X_valid = X_scaled[mask]
l_valid = labels[mask]

# Only run if more than 1 cluster found
if len(set(l_valid)) > 1:
    sil = silhouette_score(X_valid, l_valid)
    db_score = davies_bouldin_score(X_valid, l_valid)
    ch_score = calinski_harabasz_score(X_valid, l_valid)
    print(f'Silhouette: {sil:.3f}')      # Higher = better
    print(f'Davies-Bouldin: {db_score:.3f}') # Lower = better
    print(f'Calinski-Harabasz: {ch_score:.0f}')# Higher = better
```

Silhouette Score

Range: -1 to +1
+1 = perfect separation
0 = overlapping clusters
-1 = wrong assignments

Best all-round metric for DBSCAN.
Exclude noise (label=-1) before computing.

Davies-Bouldin

Range: 0 to ∞ (lower = better)
Measures avg similarity between each cluster and its most similar cluster.

Sensitive to cluster geometry.
Good complement to silhouette.

Calinski-Harabasz

Range: 0 to ∞ (higher = better)
Ratio of between-cluster dispersion to within-cluster dispersion.

Fast to compute. Good for comparing ϵ /MinPts grid search results.

Noise Rate

$\text{noise_rate} = n_{\text{noise}} / n_{\text{total}}$

High noise rate (>30%) $\rightarrow \epsilon$ too small
Zero noise rate $\rightarrow \epsilon$ too large

Target: 5-15% noise for most real datasets.

💡 Grid search tip: loop over $\epsilon \in [0.1, 0.5]$ and MinPts $\in [3, 10]$, record silhouette score for each valid (>1 cluster) result, pick the best.

Limitations & When to Use DBSCAN

Know when DBSCAN shines — and when to reach for something else

✓ USE DBSCAN WHEN...

Arbitrary shapes

Clusters are non-convex: crescents, rings, L-shapes, spirals.

Unknown K

You don't know (and don't want to guess) the number of clusters.

Outlier detection

Noise points need to be identified as a by-product of clustering.

Spatial/geo data

GPS coordinates, geographic regions, 2D/3D spatial point sets.

Low–medium dims

Typically works well for 2–10 dimensional data.

Dense regions

Data has meaningful dense regions separated by sparse gaps.

✗ LIMITATIONS

Varying density

If clusters have very different densities, a single ϵ cannot capture all of them. Use HDBSCAN instead.

Curse of dimensionality

In high-dimensional spaces (>20 dims), distance metrics become meaningless. All points look equidistant.

ϵ sensitivity

Small changes in ϵ can dramatically change results. Always validate with k-distance plot.

Large datasets

Without spatial indexing, $O(n^2)$ complexity. Use `algorithm='ball_tree'` or `'kd_tree'`.

Boundary point ambiguity

Border points can belong to multiple clusters depending on processing order.

Beyond DBSCAN — HDBSCAN

Hierarchical DBSCAN: variable density clusters, one fewer parameter

```
# Option 1: hdbscan library
import hdbscan
clusterer = hdbscan.HDBSCAN(
    min_cluster_size=10, # minimum cluster size
    min_samples=5,      # core point threshold
    cluster_selection_epsilon=0.0
)
labels = clusterer.fit_predict(X_scaled)

# Option 2: sklearn (v1.3+)
from sklearn.cluster import HDBSCAN
hdb = HDBSCAN(min_cluster_size=10)
labels = hdb.fit_predict(X_scaled)

# HDBSCAN extras
print(clusterer.probabilities_) # soft assignment
print(clusterer.outlier_scores_) # anomaly score per point
clusterer.condensed_tree_.plot() # visualise hierarchy
```

How HDBSCAN Improves on DBSCAN

Varying density ✓

Builds a hierarchy of clusters at ALL density levels, then extracts the most stable ones. No single ϵ needed.

One fewer param

Only `min_cluster_size` required. No ϵ to tune. `min_samples` is optional (defaults to `min_cluster_size`).

Soft clustering

`probabilities_` gives a [0,1] confidence score for each point's cluster membership. Useful for ambiguous points.

Outlier scores

`outlier_scores_` ranks every point by how likely it is to be noise. Use as an anomaly detection score directly.

Condensed tree

Visualise the full cluster hierarchy. Shows how clusters merge/split as density threshold varies.

When to prefer

Always try HDBSCAN first on real data. Fall back to DBSCAN when you know ϵ from domain knowledge.

📦 Install: `pip install hdbscan` OR `use sklearn.cluster.HDBSCAN` (scikit-learn ≥ 1.3 , released 2023)

Real-World Applications

Where DBSCAN powers production systems today



Geospatial Analysis

- Urban hotspot mapping: cluster GPS crime/accident coordinates to identify problem zones
- Satellite imagery: group land-use patches by spectral density
- Traffic analysis: detect congestion zones from vehicle GPS streams
- Uber/Lyft: cluster pickup locations to optimise driver positioning



Fraud & Anomaly Detection

- Credit card fraud: normal transactions cluster tightly; fraud = noise points
- Network intrusion: malicious traffic patterns appear as sparse outliers
- Insurance fraud: cluster claims by geography/type; noise = suspicious outliers
- IoT sensor faults: anomalous sensor readings isolated as noise



Medical & Bioinformatics

- Medical imaging: segment tumour regions in MRI/CT scans by density
- Single-cell RNA-seq: cluster cells by gene expression without knowing cell types
- Epidemiology: detect disease outbreak clusters in geographic patient data
- Drug discovery: cluster chemical compound similarity spaces



Science & Engineering

- Astronomy: SDSS survey uses DBSCAN to discover galaxy clusters
- Particle physics: cluster detector hits to reconstruct particle trajectories
- Manufacturing: detect surface defect clusters from sensor/camera data
- Social networks: community detection in sparse interaction graphs

Summary & Key Takeaways

What every data scientist should know about DBSCAN



Two params ϵ (radius) + MinPts (density threshold) — nothing more.



Three types Core, Border, Noise. Noise = label -1 = outlier.



No K needed Cluster count emerges from the data automatically.



Arbitrary shapes Crescents, rings, L-shapes — all handled correctly.



Scale first! ALWAYS StandardScaler before DBSCAN. Non-negotiable.



k-dist graph Use to pick ϵ — find the elbow, that's your value.



HDBSCAN next Varying density? Switch to HDBSCAN. Same ideas, smarter.



Real impact Geospatial, fraud, medical, astronomy — in production now.

Advanced Next Steps

OPTICS

Ordering Points To Identify the Clustering Structure. Generalises DBSCAN to varying density without a fixed ϵ .

HDBSCAN

Hierarchical DBSCAN. Variable density, soft membership scores, outlier scores. Preferred modern alternative.

Mean Shift

Finds cluster centres by gradient ascent on the kernel density estimate. No K, no ϵ — just bandwidth.

Spectral Clustering

Uses graph Laplacian eigenvectors. Excellent for ring/concentric structures. Needs K.

Gaussian Mixture Models

Probabilistic soft clustering. Each point gets a probability of belonging to each cluster.